ПРАКТИЧЕСКИЙ ДОКЛАД · OPEN SOURCE / .NET

От формулы к собственному языку.

Как собирать DSL из переиспользуемых компонентов — и проверять их совместимость до запуска.

Михаил Разаков / ФКН ВШЭ

ГЛАВНАЯ ПРОБЛЕМА

Языки разные. Инфраструктура повторяется.

Парсер · семантика · backend · диагностика

Формула превращается в типизированную .NET-функцию.

Сначала проверяем выражение. Затем один раз компилируем и вызываем как обычный делегат.

samples/Wist.RolloutScoring/Program.cs · сокращённый реальный пример

```
using var rules = WistEngine.CreateRestrictedArithmetic();
var check = rules.Validate(formula, inputs);
if (!check.IsValid) throw new InvalidOperationException();

var compiled = rules.Compile<Func<double, double, double, double>>(
    formula, "usage", "reliability", "incidents");
var score = compiled.CompiledDelegate(100, 90, 1);
```

01 / VALIDATE

Диагностика до исполнения

02 / COMPILE

Получаем типизированный `Func<...>`

03 / INVOKE

Вызываем многократно

Пользовательскому языку не нужен весь C#.

У диалекта собственный набор возможностей. Он зависит от задачи приложения.

РАЗРЕШАЕМ

```
2 + 3 * 4
```

Числовые константы

Арифметические операции

Вводимые параметры можно передавать из хост-приложения.

НЕ ВКЛЮЧАЕМ

```
let score = ...
```

Объявления переменных в языке

C# interop и вызовы методов хоста

Не нужен «ещё один C#» внутри продукта.

> Ограничение синтаксиса — это контроль возможностей, но не полная изоляция недоверенного кода.

Четыре строки задают набор возможностей языка.

Собственный диалект не требует переписывать существующий парсер.

shop-rules.wistdialect / файл из демонстрации

```
dialect ShopRules
use Arithmetic,Numbers,
Scopes,Whitespaces
backend cil,interpreter
security restricted
```

use

Нужные модули языка

backend

Разрешённые маршруты

security

Профиль конфигурации, *не* песочница

Один диалект — два маршрута исполнения.

Один вход
ShopRules

2 + 3 * 4

interpreter	Удобен для диагностики, тестов и сопоставления поведения
CIL	Подготавливает исполняемый код для многократных вызовов из .NET

Проверенный пример: оба маршрута вычислили выражение одинаково.

Ускорение CIL здесь не измерялось; универсальная эквивалентность не доказана.

Несовместимость обнаруживается до исполнения программы.

ДИАЛЕКТ РАЗРЕШАЕТ

```
backend interpreter
```

ПРИЛОЖЕНИЕ ЗАПРАШИВАЕТ

```
BackendId = "cil"
```

ФАКТИЧЕСКОЕ СООБЩЕНИЕ ОБ ОШИБКЕ CLI

```
Wist dialect 'ShopRulesDebug' does not enable backend 'cil'.
Enabled backends: interpreter.
```

Чтение диалекта → Проверка backend ✕ → ~~Создание LanguageRuntime~~ → ~~Выполнение~~

LanguagePlan фиксирует выбор компонентов до запуска.

LanguageDefinition
Модули, зависимости, ограничения и backend

↓

LanguageCompiler
Разрешает зависимости и проверяет совместимость

↓

LanguagePlan
Зафиксированный состав и порядок компонентов

↓

LanguageRuntime
Исполняющая сессия выбранного конвейера

ГЛАВНЫЙ МЕХАНИЗМ

«Сначала договориться, что можно собрать.
Потом исполнять».

LanguageRuntime связывает компоненты по плану: разбор, преобразования, backend. Это больше, чем виртуальная машина.

Когда UT оправдан, а когда достаточно обычного кода?

НЕ НУЖЕН ДОПОЛНИТЕЛЬНЫЙ FRAMEWORK

Один фиксированный сценарий

Небольшая формула или один парсер

Не требуются варианты диалектов

Нет независимых модулей и backend

Проще оставить обычный C# или готовую библиотеку.

UNIVERSALTOOLCHAIN ИМЕЕТ СМЫСЛ

Семейство близких DSL

Повторное использование модулей

Разные наборы конструкций и backend

Важна ранняя проверка всей сборки

Платим за инфраструктуру, когда она устраняет повторение.

08 / ДВА УРОВНЯ АБСТРАКЦИИ

Feature и семантическое ядро

Создавать язык проще, а модули — независимее друг от друга.

ДЛЯ СОЗДАТЕЛЯ ЯЗЫКА — **Уже в SDK**

Feature

Вместо ручного подбора разнородных модулей выбираем нужную возможность языка.

Feature: «Оптимизации» — Один запрос

↓

LanguageCompiler подбирает совместимые реализации

Interpreter	CIL / .NET	Новый backend
свои оптимизации	свои оптимизации	если есть реализация

ДЛЯ РАЗРАБОТЧИКА МОДУЛЯ — **Направление развития**

Семантическое ядро

Модули обмениваются информацией через общие контракты, не зная реализаций друг друга.

Модуль типов
публикует TypeOf(x)

→

Общие факты
типы, свойства, ограничения

→

Оптимизатор
запрашивает TypeOf(x)

Нет прямого импорта: оптимизатор работает с контрактом, а не с конкретным модулем типизации.

Feature выбирает состав языка. **Семантическое ядро** помогает модулям обмениваться знаниями без прямых зависимостей.

ТРИ ИДЕИ

Собирайте язык. Не пересобирайте весь компилятор.

Выражение можно проверять и компилировать. Конструкции — выбирать модульно. Конфигурацию — проверять до запуска.

Спасибо! Буду рад вопросам.



Исходный код

GitHub · UniversalToolchain



Документация диалектов

Dialect Files · GitHub